

Problem Set 1



Applied Stats II
Shelly Veal-Upham
25337422

Due: February 11, 2026

Question 1

The Kolmogorov-Smirnov test uses cumulative distribution statistics test the similarity of the empirical distribution of some observed data and a specified PDF, and serves as a goodness of fit test. The test statistic is created by:

$$D = \max_{i=1:n} \left\{ \frac{i}{n} - F_{(i)}, F_{(i)} - \frac{i-1}{n} \right\}$$

where F is the theoretical cumulative distribution of the distribution being tested and $F_{(i)}$ is the i th ordered value. Intuitively, the statistic takes the largest absolute difference between the two distribution functions across all x values. Large values indicate dissimilarity and the rejection of the hypothesis that the empirical distribution matches the queried theoretical distribution. The p-value is calculated from the Kolmogorov- Smirnov CDF:

$$p(D \leq d) = \frac{\sqrt{2\pi}}{d} \sum_{k=1}^{\infty} e^{-(2k-1)^2 \pi^2 / (8d^2)}$$

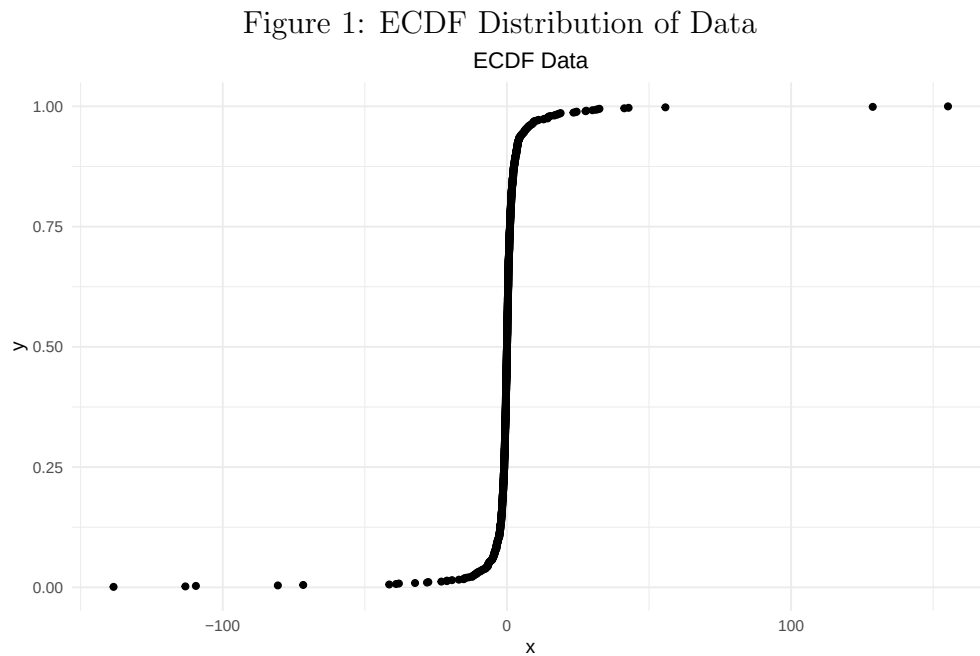
which generally requires approximation methods (see Marsaglia, Tsang, and Wang 2003). This so-called non-parametric test (this label comes from the fact that the distribution of the test statistic does not depend on the distribution of the data being tested) performs poorly in small samples, but works well in a simulation environment. Write an R function that implements this test where the reference distribution is normal. Using R generate 1,000 Cauchy random variables (`rcauchy(1000, location = 0, scale = 1)`) and perform the test (remember, use the same seed, something like `set.seed(123)`, whenever you're generating your own data).

First, let's take a look at the data:

```

1 set.seed(123)
2 # create empirical distribution of observed data
3 data_1 <- rcauchy(1000, location = 0, scale = 1)
4 ECDF <- ecdf(data_1)
5 empiricalCDF <- ECDF(data_1)
6 # generate test statistic
7 D <- max(abs(empiricalCDF - pnorm(data_1)))
8
9 df <- data.frame(
10   x = data_1,
11   y = empiricalCDF
12 )
13
14 ECDF_plot <- ggplot(df, aes(x = x, y = y)) + # Visualizing
15   geom_point() +
16   labs(title = "ECDF Data") +
17   theme(plot.title = element_text(hjust = 0.5))

```



The range on the x axis is from the negative hundreds to the positive hundreds, while y only spans from 0 to 1. This makes sense, as the cumulative distribution is made up of probabilities (ranging 0:1). Knowing that there are different kinds of cumulative distributions, I developed the Kolmogorov-Smirnov test by hand with the intention of making it at least a little more flexible than only working with the Empirical CDF:

```

1 KS_test <- function(F_Fun, x) {
2
3   if (identical(F_Fun, ecdf)) {
4     x <- sort(x)
5     n <- length(x)
6     i <- seq_along(x)
7     F_inter <- F_Fun(x)
8     FVals <- F_inter(x)
9   }
10  else {
11    n <- length(x)
12    i <- seq_along(x)
13    FVals <- F_Fun(x)
14  }
15
16  D <- max(abs(FVals - pnorm(x)))
17  p_val <- (sqrt(2*pi)/D)*sum(exp(-(2*i-1)^2*pi^2)/(8*(D^2)))
18
19  return(list(
20    D = D,
21    p_value = p_val))
22 }
23
24 KS_test(F_Fun = ecdf, x = data_1)
25 ks.test(data_1, "pnorm")

```

As expected, the outputs of my KS test function and the built-in `ks.test()` are nearly identical:

```

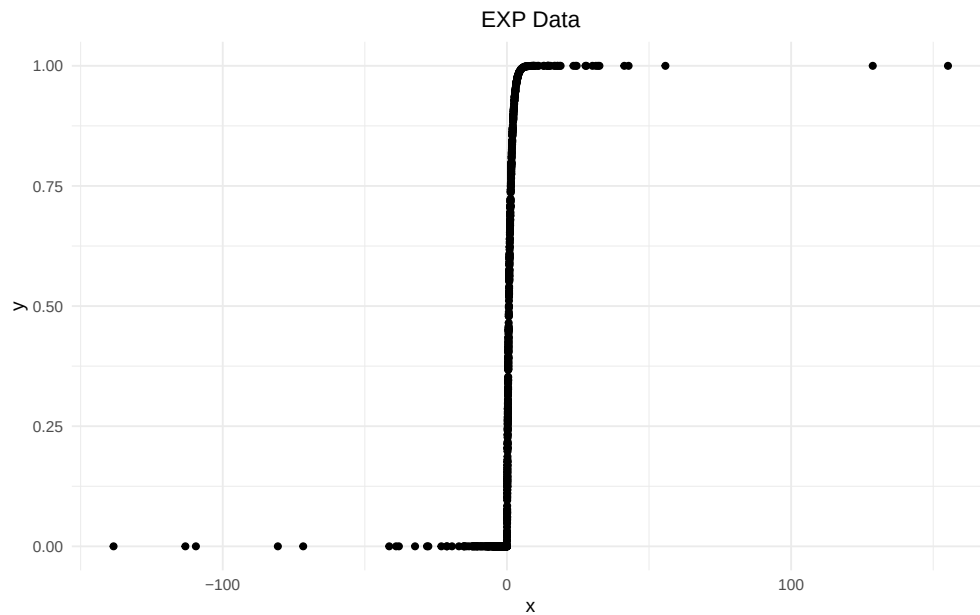
KS_test(): D = 0.1347, p = 0.0066
ks.test(): D = 0.1357, p = 2.2e-16

```

There is a notable difference in the p-values— the built-in `ks.test()` results in a p-value much closer to 0 than my function, which is likely a result of my crude approximation of the infinite sum using a sum of only the length of the data.

Perhaps in futility I decided to create my KS test function such that it might accommodate other Cumulative Distribution Functions besides ECDF, such as `pexp()`:

Figure 2: ECDF Distribution of Data



```

1 pexp_y <- pexp(data_1)
2
3 df_pexp <- data.frame(
4   x = data_1,
5   y = pexp_y
6 )
7
8 exp_plot <- ggplot(df_pexp, aes(x = x, y = y)) + # Visualizing
9   geom_point() +
10  labs(title = "EXP Data") +
11  theme(plot.title = element_text(hjust = 0.5))
12
13 KS_test(F_Fun = pexp, x = data_1)
14 ks.test(data_1, pexp)

```

Again, the outputs are nearly identical:

```

KS_test(): D = 0.4993, p = 0.0001
ks.test(): D = 0.4930, p = 2.2e-16

```

Question 2

Estimate an OLS regression in R that uses the Newton-Raphson algorithm (specifically BFGS, which is a quasi-Newton method), and show that you get the equivalent results to using `lm`.

Utilizing the tutorial material from this week as a rough template, I developed the following code for my OLS by hand:

```
1 set.seed (123)
2 data_2 <- data.frame(x = runif(200, 1, 10))
3 data_2$y <- 0 + 2.75*data_2$x + rnorm(200, 0, 1.5)
4
5 squared_errors <- function(outcome, input, parameter) {
6   input_mat <- as.matrix(input)
7   n <- nrow(input_mat)
8   k <- ncol(input_mat)
9   beta <- parameter[1:k]
10  sum_sq_e <- sum((outcome - input_mat%*%beta)^2)
11  return(sum_sq_e)
12 }
13
14 results_OLS <- optim(fn=squared_errors, outcome=data_2$y,
15                    input=cbind(1, data_2$x), par=c(1,1),
16                    hessian=T, method="BFGS")
17
18 results_OLS$par
19 lm(data_2$y ~ data_2$x)
```

The output of my function and the `lm()` function are identical, and well approximate the β values we would expect from our original formula (seen in line 3 of the above R code):

`results_OLS$par`: $\beta_0 = 0.1392, \beta_1 = 0.2.727$
`lm(data_2$y ~ data_2$x)`: $\beta_0 = 0.1392, \beta_1 = 0.2.727$